Fase 1: Grupo 7

Smart Supply Chain Risk AI (Torre de Control Inteligente)

MATEMALAS MANZANARO, Adrián
ROMERO MONTORO, Raúl
RINCÓN ORTIZ, Raúl
Curso de Especialización de Inteligencia Artificial y Big Data
Proyecto de big data e inteligencia artificial

Índice

1. Título del proyecto + integrantes del equipo....3
2. Resumen ejecutivo....3
3. Descripción funcional del sistema....3
4. Alcance definitivo (IN / OUT)....4
5. Arquitectura del sistema (alto nivel)....4
6. Tecnologías seleccionadas....6
7. Modelo de datos....8
8. Datos....10
9. Ética, legalidad y accesibilidad....12
10. Backlog inicial....13
11. Plan de trabajo....16
12. Estrategia de desarrollo....16
13. Riesgos técnicos actualizados....17
14. Repositorio del proyecto....19

1. Título del proyecto + integrantes del equipo

- Título del proyecto: Smart Supply Chain Risk AI
- Integrantes:
    - Adrián Matemalas Manzanaro
    - Raúl Romero Montoro
    - Raúl Rincón Ortiz

2. Resumen ejecutivo

Sistema B2B de analítica predictiva en tiempo real para anticipar demoras en envíos. Integra ML (XGBoost/LightGBM) mediante API REST en FastAPI y un dashboard en Streamlit ("Torre de Control") para optimizar rutas y reducir penalizaciones de SLA.

3. Descripción funcional del sistema

Plataforma B2B de analítica predictiva en tiempo real que evalúa el riesgo de demoras en envíos mediante Machine Learning (XGBoost y LightGBM). Expone sus servicios a través de una API REST en FastAPI y ofrece visualización operativa mediante una dashboard ("Torre de Control") en Streamlit.

Flujo General del Sistema

- **1. Ingesta:** Recepción de datos de envíos vía HTTP en la API de FastAPI.
- **2. ETL y Feature Engineering:** Validación con Pydantic, limpieza y generación de variables clave.
- **3. Inferencia:** Ejecución de modelos ML (.joblib) para estimar probabilidad de retraso y nivel de riesgo.
- **4. Visualización y Alertas:** Muestra de resultados en Streamlit y envío de notificaciones por Webhooks ante riesgos críticos.

Casos de Uso Principales

- **Gestor Logístico:** Monitorea envíos en el dashboard, evalúa riesgos y simula rutas alternativas.
- **Data Engineer / MLOps:** Supervisa el pipeline ETL, gestiona reentrenamientos y compara XGBoost vs. LightGBM.
- **Responsables de Operaciones:** Reciben alertas automáticas vía Webhooks ante envíos de alto riesgo.

Capacidades del Sistema (Límites Funcionales)

- **SÍ incluye:** Pipelines ETL/Feature Engineering, inferencia con XGBoost/LightGBM, API REST con FastAPI, dashboard interactivo con simulador y alertas por Webhooks en Streamlit, y dockerización básica.
- **NO incluye:** Facturación o pasarelas de pago, integración directa con hardware/dispositivos IoT, autenticación avanzada (SSO) ni canalizaciones de CI/CD productivas.

4. Alcance definitivo (IN / OUT)

- **Alcance IN:** Pipeline ETL/Feature Engineering, modelos XGBoost/LightGBM, API FastAPI y dashboard Streamlit con simulador y alertas Webhooks.
- **Alcance OUT:** Facturación, hardware IoT, autenticación SSO y CI/CD en producción.

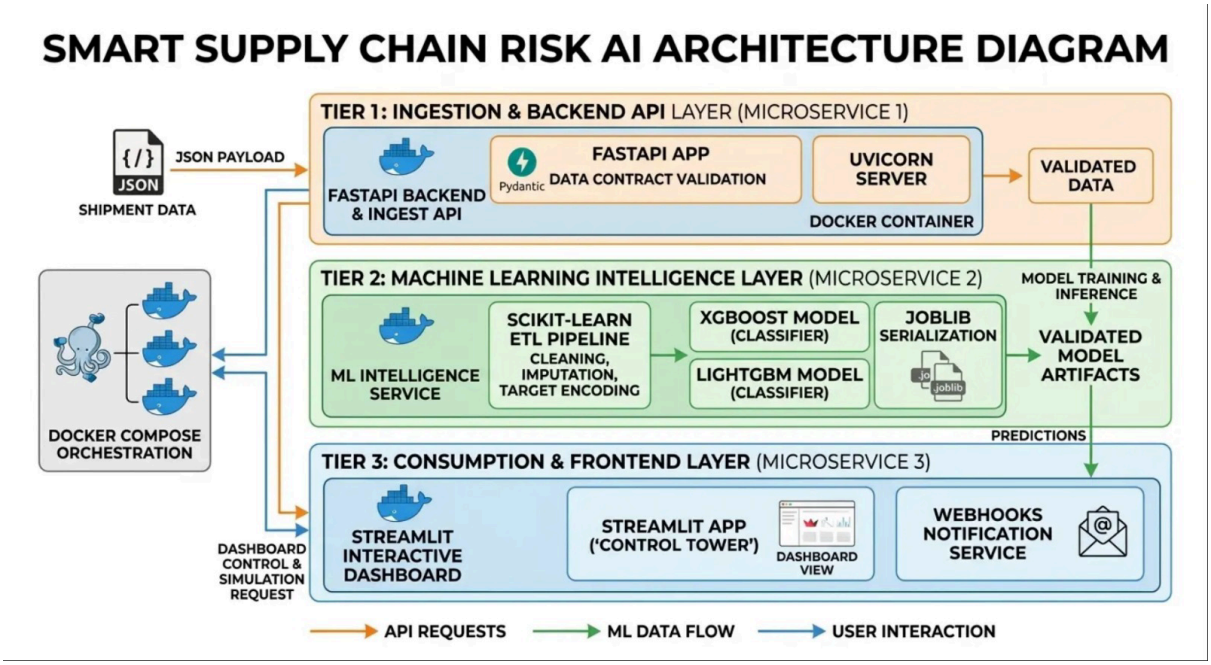
5. Arquitectura del sistema (alto nivel)

El sistema Smart Supply Chain Risk AI está diseñado bajo una arquitectura modular y desacoplada orientada a microservicios en tres capas principales (Ingesta/Servicio, Inteligencia y Consumo), orquestadas de forma aislada e independiente:

- **Frontend (Streamlit – Interfaz de usuario y Torre de Control):** Desarrollado en Streamlit, constituye el punto de interacción para gestores logísticos y analistas. Ofrece un cuadro de mando interactivo ("Torre de Control") que consulta asíncronamente el backend mediante llamadas HTTP REST para mostrar visualizaciones, mapas de riesgo y KPIs operativos.
- **Backend (FastAPI y validación con Pydantic):** Construido con FastAPI sobre servidor ASGI (Uvicorn). Gestiona los contratos de datos de entrada en tiempo real mediante esquemas Pydantic (`ShipmentIngestSchema`), garantizando la validación estricta de tipos, formatos y rangos antes de la inferencia, y generando documentación OpenAPI interactiva.
- **Base de datos (Estructura relacional):** Organiza la información en un esquema relacional optimizado compuesto por cuatro tablas principales: `Orders` (transacciones comerciales), `Shipments` (bultos e ítems

despachados), `Routes` (corredores logísticos y zonas de destino agrupadas por Market, Order_City y Shipping_Mode) y `Predictions_Log` (auditoría e historial de inferencias realizadas por el modelo ML).

- **Servicios ML (Modelos XGBoost y LightGBM):** Modelos de Gradient Boosting supervisados optimizados para clasificación tabular. Se serializan mediante `.joblib` e integran en la memoria RAM del servicio backend mediante el evento `lifespan` de FastAPI, garantizando inferencias de muy baja latencia.
- **Pipeline de datos (ETL con Pandas y Scikit-learn):** Canalización automatizada para la ingesta, limpieza de atributos con fuga de datos (Data Leakage), imputación de valores ausentes y transformación de variables categóricas de alta cardinalidad mediante Target Encoding dentro de Pipelines Scikit-learn reutilizables.
- **Dashboards (Panel interactivo y simulador "What-If"):** Sistemas de supervisión dentro de Streamlit que integran métricas clave de rendimiento logístico y una herramienta de simulación interactiva ("What-If") que permite ajustar parámetros de ruta en tiempo real para reevaluar instantáneamente el nivel de riesgo predictivo.
- **Otros servicios (Alertas por Webhooks y Docker Compose):** Servicio asíncrono de notificaciones automáticas mediante llamadas HTTP POST (Webhooks) disparadas al detectar envíos de alto riesgo, y empaquetado integral de microservicios con Docker y Docker Compose para garantizar el aislamiento, la portabilidad y la reproducibilidad del entorno.



6. Tecnologías seleccionadas

Para la construcción del sistema en sus tres capas (Ingesta/Servicio, Inteligencia y Consumo), se han seleccionado las siguientes tecnologías:

- **Python (v3.10+)**
  - *Por qué se ha elegido:* Estándar consolidado en ciencia de datos, machine learning y desarrollo backend ligero. Ofrece soporte maduro para manipulación de datos, entrenamiento e inferencia en producción.
  - *Alternativa descartada:* Java o Scala, descartados por aportar mayor complejidad y sobrecarga de desarrollo para los plazos del proyecto.
- **FastAPI + Pydantic**
  - *Por qué se ha elegido:* Framework backend asíncrono de alto rendimiento con validación estricta y automática de esquemas de datos entrantes mediante modelos Pydantic, generando documentación OpenAPI nativa.
  - *Alternativa descartada:* Flask, descartado por requerir validación manual de esquemas y ofrecer menor rendimiento asíncrono.
- **XGBoost y LightGBM**

    - *Por qué se ha elegido:* Algoritmos basados en árboles de decisión (*Gradient Boosting*) optimizados para clasificación tabular. Destacan por su velocidad de inferencia, capacidad para manejar datos heterogéneos y soporte eficiente para *Threshold Tuning*.
    - *Alternativa descartada:* Redes Neuronales Profundas (PyTorch/TensorFlow), descartadas por demandar mayor volumen de datos, menor interpretabilidad operativa y un coste computacional desproporcionado.
- **Scikit-learn y Pandas**
    - *Por qué se ha elegido:* Pandas permite la carga, análisis y transformaciones estructuradas en memoria. Scikit-learn proporciona los pipelines de preprocesamiento, codificación categórica y cálculo de métricas de validación (Recall, F1-score, ROC-AUC).
    - *Alternativa descartada:* Apache Spark / PySpark, descartado porque el volumen del dataset logístico se procesa eficientemente en memoria local sin requerir computación distribuida.
- **Joblib**
    - *Por qué se ha elegido:* Serialización rápida y eficiente de modelos y pipelines de Scikit-learn/XGBoost para su posterior carga en memoria RAM en el arranque de la API.
    - *Alternativa descartada:* Pickle estándar, por ser menos óptimo en la persistencia de matrices numéricas grandes.
- **Streamlit**
    - *Por qué se ha elegido:* Permite desarrollar de forma ágil cuadros de mando interactivos ("Torre de Control"), simulaciones visuales de rutas y cuadros de mando en Python puro.
    - *Alternativa descartada:* React o Dash, descartados por incrementar la sobrecarga en el desarrollo de interfaces desacopladas.
- **Docker y Docker Compose**
    - *Por qué se ha elegido:* Empaquetado y aislamiento de los microservicios (API FastAPI y frontend Streamlit), asegurando portabilidad y reproducibilidad del entorno entre los integrantes del equipo.

Alternativa descartada: Despliegue en máquina virtual directa, por falta de aislamiento de librerías y menor facilidad de réplica.

7. Modelo de datos

El sistema organiza los datos logísticos y los resultados de inferencia en cuatro entidades operativas principales:

Tablas y Claves

Orders (orders): Registra la información de cada orden de compra.

- **Clave primaria:** Order_Id.
- **Clave foránea**: Customer_Id.
- **Campos principales:** Order_Region, Order_Country, Order_Date, Customer_Segment, Order_Status.

Shipments (shipments): Registra cada ítem o envío asociado a una orden.

- Clave primaria: Order_Item_Id.
- Claves foráneas: Order_Id (relación N:1 con Orders) y Route_Id (relación N:1 con Routes).
- Campos principales: Days_for_shipment_scheduled, Shipping_Mode, Scheduled_Shipping_Date, Actual_Shipping_Date, Product_Category_Id, Department_Id, Late_delivery_risk.

Routes (routes): Registra las rutas logísticas asociadas a los envíos.

- Clave primaria: Route_Id.
- Campos principales: Market, Order_City, Shipping_Mode, Estimated_Transit_Days, Historical_Delay_Rate.

Predictions_Log (predictions_log): Registra la auditoría de las predicciones realizadas por el modelo de Machine Learning.

- Clave primaria: Prediction_Id.
- Clave foránea: Order_Item_Id (relación N:1 con Shipments).
- Campos principales: Prediction_Timestamp, Late_Risk_Probability, Risk_Level, Model_Version.

Relaciones

- Una orden de compra genera uno o más ítems de envío (Orders 1 a N Shipments), mediante Order_Id.
- Cada envío transita por una ruta logística catalogada (Routes 1 a N Shipments), mediante Route_Id.

- Cada envío evaluado por el modelo registra una o más auditorías de inferencia a lo largo del tiempo (Shipments 1 a N Predictions_Log), mediante Order_Item_Id.

Diferencia entre modelo de datos (BD) y features del modelo ML

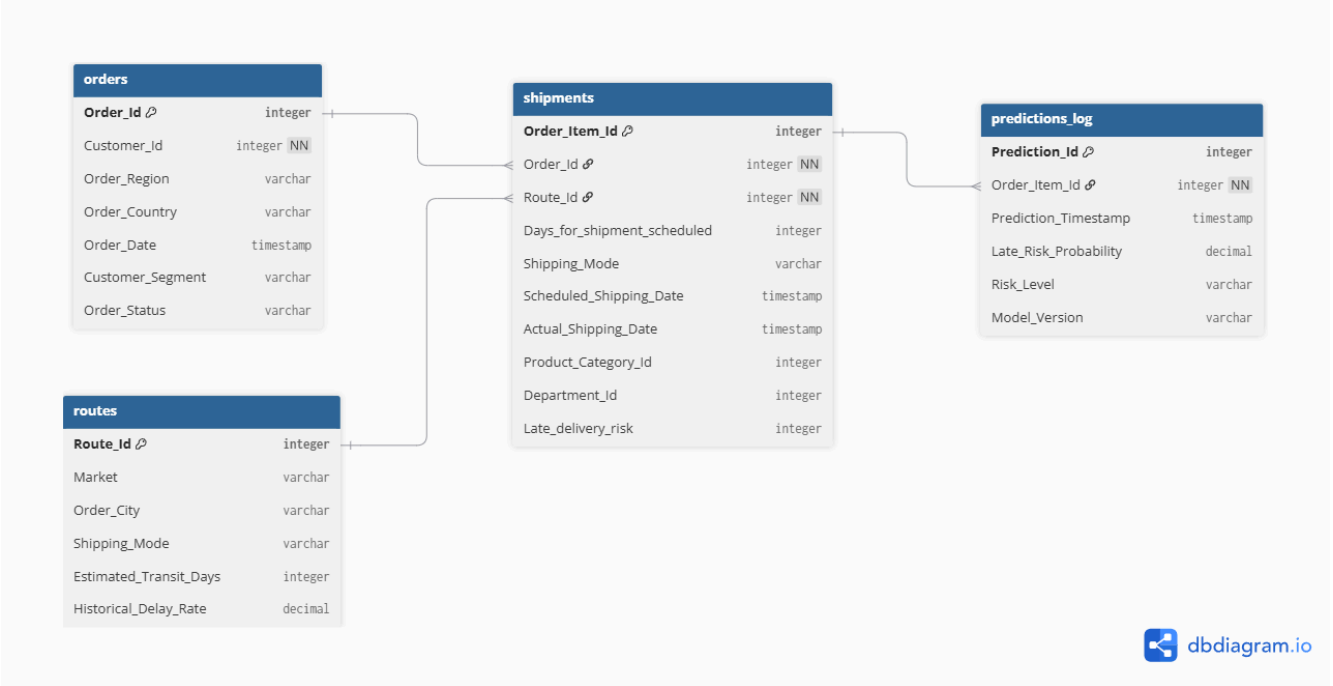
Es importante distinguir entre el modelo de datos almacenado en la base de datos relacional y el conjunto de variables (features) suministradas al modelo de Machine Learning:

- Variable Target: La columna Late_delivery_risk actúa como variable objetivo (target) durante el entrenamiento. No se utiliza como feature de entrada del modelo.
- Prevención de Data Leakage: Variables como Actual_Shipping_Date y otros atributos conocidos únicamente después del instante T0 (confirmación del pedido) se almacenan en la base de datos con fines históricos y transaccionales, pero se excluyen de las entradas del modelo predictivo para evitar filtraciones de información futura.

Diccionario de Variables Clave (para inferencia y entrenamiento)

- **Late Delivery Risk** (Entero: 0 o 1): Variable objetivo (target) del modelo. Indica si el envío presenta riesgo de retraso.
- **Days_for_shipment_scheduled** (Entero): Número de días comprometidos para el envío según el plazo programado.
- **Shipping_Mode** (Texto): Modalidad de envío contratada, como Standard Class, First Class, Second Class o Same Day.
- **Order_Region** (Texto): Región geográfica de destino de la orden.
- **Order_Country** (Texto): País de destino de la orden.
- Order_Date (Fecha y hora): Momento en el que se registra la orden. Puede utilizarse para obtener variables temporales y analizar la estacionalidad.
- **Customer_Segment** (Texto): Segmento al que pertenece el cliente.
- **Order_Status** (Texto): Estado de la orden. Se almacena en la base de datos, pero debe excluirse del modelo si su valor solo se conoce después del instante T0.
- **Product_Category_Id** (Entero): Identificador de la categoría del producto enviado.
- **Department_Id** (Entero): Identificador del departamento asociado al producto o envío.

- **Market** (Texto): Mercado o zona geográfica asociada a la ruta logística.
- **Order_City** (Texto): Ciudad asociada al destino de la orden.
- **Estimated_Transit_Days** (Entero): Número estimado de días necesarios para completar el tránsito de la ruta.
- **Historical_Delay_Rate** (Decimal): Tasa histórica de retrasos de la ruta, calculada a partir de incidencias anteriores.
- **Late_Risk_Probability** (Decimal: 0.0–1.0): Probabilidad estimada por el modelo de que se produzca un retraso.
- **Risk_Level** (Texto): Nivel de riesgo calculado a partir de la probabilidad, clasificado como Bajo, Medio o Alto.
- **Prediction_Timestamp** (Fecha y hora): Momento exacto en el que se realiza la predicción.
- **Model_Version** (Texto): Versión del modelo utilizada para realizar la inferencia, por ejemplo, xgboost_v1.2.



8. Datos

- **Fuente:** Dataset estructurado **DataCo Global Supply Chain Intelligence** (almacenado localmente en el directorio /data y obtenido de repositorios abiertos para análisis de cadena de suministro).

- **Licencia: CC0 1.0 (Public Domain)**, según la ficha del dataset publicada en Kaggle. Esta licencia permite la reutilización, modificación y procesamiento de los datos, resultando adecuada para su utilización en el contexto académico y experimental del proyecto.
- **Tipo de datos:** Datos tabulares estructurados en formato CSV. Contienen atributos temporales (fechas de orden y despacho programado), categóricos (regiones, países, modalidades de envío) y numéricos discretos/continuos (cantidades, tiempos planificados).
- **Volumen estimado:** Aproximadamente **180.519 registros** y **53 columnas** originales (~100 MB de tamaño de archivo).
- **Frecuencia de actualización:** Ingesta estática por lotes (*batch*) para entrenamiento e ingeniería de características, e ingesta transaccional simulada registro a registro vía llamadas HTTP a la API REST para inferencias en tiempo real.
- **Naturaleza de los datos:**
    - **Reales anonimizados:** Datos históricos de transacciones logísticas y cadenas de suministro operativas.
    - **Simulados:** Parámetros y combinaciones de ruta introducidos manualmente por el usuario desde la interfaz de Streamlit para pruebas de estrés y simulación de escenarios alternativos.
    - *Nota formal:* **No se emplean datos mockeados sintéticamente desde cero**; la base empírica reside en los registros históricos depurados.
- **Calidad esperada y tratamiento inicial:** Limpieza de registros redundantes, eliminación de atributos sin valor predictivo o PII (nombres de clientes, correos, descripciones no estructuradas) e imputación/codificación de variables categóricas mediante Target Encoding.
- **Riesgos de datos y mitigaciones:**
    - *Fuga de datos temporal (Data Leakage):* El dataset incluye atributos registrados tras el cierre del trayecto (como Days for shipping (real) o Delivery Status). Se mitiga eliminando de forma sistemática durante la ETL cualquier columna que no esté disponible en el momento cero de confirmación de la orden.
    - *Alta cardinalidad:* Dispersión en atributos geográficos (ciudades, códigos de envío). Se mitiga agrupando valores de baja frecuencia bajo la etiqueta Other y aplicando Target Encoding.

- ***Threshold Tuning y coste asimétrico:*** Dado el elevado coste operativo de los falsos negativos (un envío retrasado no detectado supone incumplimiento de SLA y penalizaciones), se aplicará *Threshold Tuning* ajustando el umbral de decisión en los modelos para maximizar el Recall (>80%) en la detección de incidencias.

Instante de inferencia y prevención de Data Leakage

- El instante de inferencia operativo se fija estrictamente en **T0 (momento de confirmación del pedido, previo al despacho del bulto)**. Para garantizar la viabilidad del modelo en un entorno de producción real y evitar cualquier fuga de datos (Data Leakage), el pipeline de preprocesamiento elimina rigurosamente todas aquellas columnas registradas a posteriori del trayecto, como *Days for shipping (real)*, *Actual Shipping Date* o *Delivery Status*, limitando el conjunto de atributos a la información disponible únicamente en T0.

9. Ética, legalidad y accesibilidad

A continuación se detallan las directrices y consideraciones éticas, legales y de accesibilidad aplicadas al proyecto Smart Supply Chain Risk AI:

- **Presencia o ausencia de datos personales (PII):** El sistema trata exclusivamente datos de carácter logístico, operativo, de rutas y de empresas B2B. No se realiza recopilación ni procesamiento de datos personales sensibles (PII) de usuarios o particulares.
- **Cumplimiento GDPR:** Se aplican de manera estricta los principios de minimización de datos y seguridad en el almacenamiento y transmisión de la información conforme a la normativa vigente (GDPR/RGPD).
- **Riesgos de sesgo:** Se contemplan posibles sesgos inherentes a datos históricos en el rendimiento de determinados transportistas, rutas específicas o zonas geográficas complejas.
- **Medidas de mitigación:** Implementación de auditorías periódicas sobre los modelos predictivos y revisión sistemática de las métricas de error y desempeño desglosadas por segmento.
- **Accesibilidad básica prevista en Streamlit:** Diseño de la interfaz visual con alto contraste de color, estructura clara de elementos e integración de compatibilidad con navegación básica por teclado y lector.

10. Backlog inicial

A continuación se detalla el Backlog Inicial del proyecto Smart Supply Chain Risk AI con sus respectivos responsables asignados:

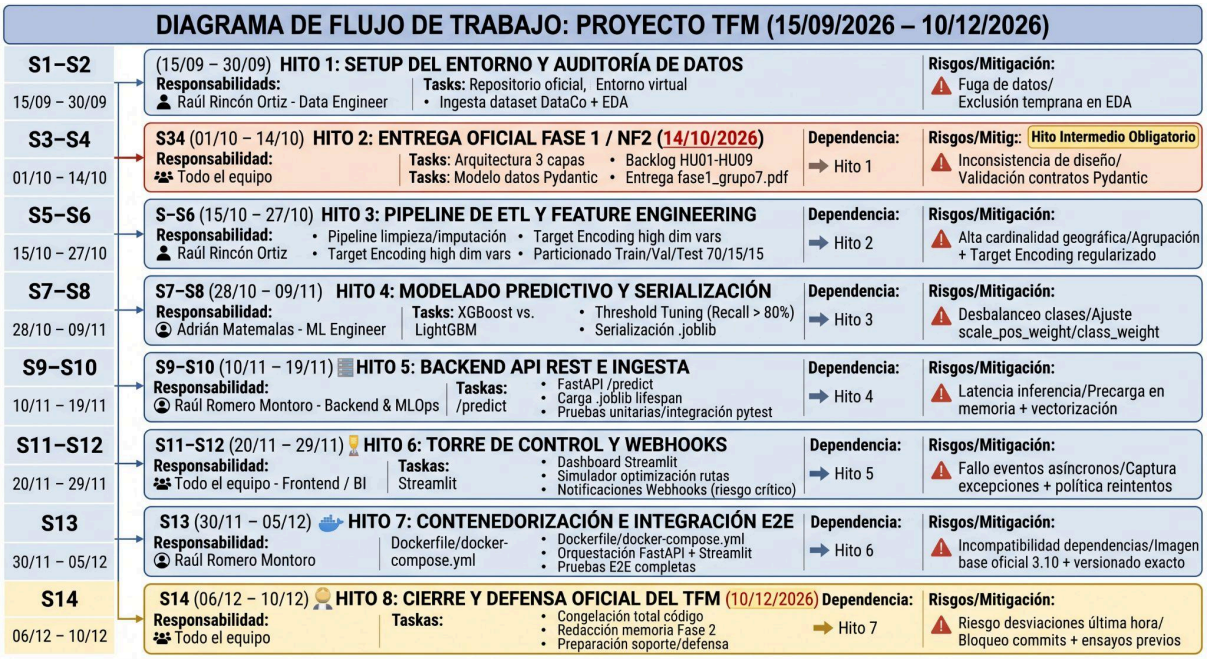
- **TASK-01: Implementación Endpoint API REST para Ingesta de Datos (FastAPI + Pydantic)**
  - **Asignado:** Raúl Romero (Backend y MLOps)
  - **Descripción:** Desarrollar el endpoint `/ingest` en la API REST con FastAPI para recibir el payload JSON con información de envíos en tiempo real y transmitirlo al pipeline de inferencia.
  - **Criterios de Aceptación:**
    - Definición del modelo Pydantic `ShipmentIngestSchema` con validación estricta de tipos de datos y campos requeridos.
    - Implementación del endpoint POST `/ingest` en FastAPI procesando correctamente las peticiones entrantes.
    - Respuesta HTTP 200 OK con confirmación en payloads válidos y HTTP 422 Unprocessable Entity en estructuras inválidas.
- **TASK-02: Construcción del Pipeline de ETL y Feature Engineering**
  - **Asignado:** Raúl Rincón (Data Eng.)
  - **Descripción:** Crear e integrar la canalización modular de transformación de datos logísticos crudos en features predictivas aptas para el modelo ML, evitando fuga de datos.
  - **Criterios de Aceptación:**
    - Limpieza de variables posteriores a la confirmación (Data Leakage) e imputación de valores faltantes.
    - Aplicación de Target Encoding y escalado de variables cuantitativas dentro de un Pipeline de Scikit-Learn.
    - Generación exitosa de matrices procesadas (Train/Val/Test 70/15/15) exportables para modelado.
- **TASK-03: Entrenamiento y Optimización del Modelo Predictivo XGBoost**
  - **Asignado:** Adrián Matemalas (ML Eng.)
  - **Descripción:** Entrenar el clasificador XGBoost utilizando el conjunto de datos procesado, priorizando la maximización del Recall sobre la clase crítica (retraso) mediante Threshold Tuning para mitigar el coste asimétrico de los falsos negativos.
  - **Criterios de Aceptación:**

        - Ajuste del umbral de decisión (Threshold Tuning) garantizando un Recall mayor al 80% en la detección de retrasos.
        - Ejecución de Threshold Tuning garantizando un Recall mayor al 80% en la clase crítica.
        - Registro y documentación de métricas de desempeño (F1–score, ROC-AUC, Matriz de Confusión).
- **TASK-04: Desarrollo Benchmark y Modelo Alternativo LightGBM**
    - **Asignado:** Adrián Matemalas (ML Eng.)
    - **Descripción:** Implementar el modelo LightGBM como alternativa predictiva para comparar latencia, precisión y eficiencia computacional frente al modelo XGBoost.
    - **Criterios de Aceptación:**
        - Entrenamiento del modelo LightGBM con soporte nativo para variables categóricas.
        - Evaluación comparativa de tiempos de inferencia (ms/petición) y consumo de memoria RAM.
        - Elaboración del informe técnico de benchmark para selección del modelo en producción.
- **TASK-05: Construcción Dashboard "Torre de Control" en Streamlit**
    - **Asignado:** Trabajo conjunto (Frontend / BI)
    - **Descripción:** Diseñar e implementar una interfaz gráfica interactiva en Streamlit que consuma la API REST para visualizar indicadores operativos y mapas de riesgo.
    - **Criterios de Aceptación:**
        - Conexión funcional mediante HTTP client con la API FastAPI backend.
        - Vista general con KPIs principales (% en riesgo, envíos críticos, tiempos promedio).
        - Implementación de filtros dinámicos por región, transportista y nivel de riesgo.
- **TASK-06: Módulo Simulador de Optimización de Rutas**
    - **Asignado:** Trabajo conjunto (Frontend / BI)
    - **Descripción:** Añadir una sección interactiva "What-If" en Streamlit para permitir a los analistas modificar parámetros de ruta o modal y recalcular instantáneamente las probabilidades de retraso.
    - **Criterios de Aceptación:**
        - Formularios e inputs interactivos para ajuste manual de variables (Shipping Mode, Region, etc.).

        - Recálculo dinámico del riesgo consumiendo el endpoint predictivo de la API.
        - Comparativa visual gráfica antes/después del escenario simulado.
- **TASK-07: Servicio de Alertas Automáticas vía Webhooks**
    - **Asignado:** Raúl Romero (Backend y MLOps)
    - **Descripción:** Desarrollar la lógica asíncrona para emitir notificaciones HTTP POST (Webhooks) hacia sistemas externos cuando una inferencia supere el umbral de riesgo alto.
    - **Criterios de Aceptación:**
        - Disparo de payload JSON de alerta al detectar `Risk_Level == "Alto"`.
        - Manejo asíncrono de reintentos y captura de excepciones en entregas fallidas.
        - Configuración modular del endpoint de destino vía variables de entorno.
- **TASK-08: Módulo de Serialización y Persistencia de Artefactos (.joblib)**
    - **Asignado:** Raúl Rincón (Data Eng.)
    - **Descripción:** Establecer el pipeline de exportación y carga eficiente de artefactos ML (modelos + encoders) en formato `.joblib`, integrándolos en el arranque de FastAPI.
    - **Criterios de Aceptación:**
        - Función estandarizada de guardado de modelos/pipelines utilizando `joblib.dump`.
        - Precarga de artefactos en memoria RAM mediante el evento `lifespan` de FastAPI.
        - Verificación de retrocompatibilidad y validación de tipos tras la deserialización.
- **TASK-09: Dockerización y Orquestación de Microservicios con Docker Compose**
    - **Asignado:** Raúl Romero (Backend y MLOps)
    - **Descripción:** Crear las imágenes Docker y la configuración de Docker Compose para desplegar el backend (FastAPI) y frontend (Streamlit) en un entorno containerizado y reproducible.
    - **Criterios de Aceptación:**
        - Creación de `Dockerfile` optimizado basado en Python 3.10-slim para API y Dashboard.
        - Configuración de `docker-compose.yml` conectando los contenedores mediante red interna.

- Despliegue correcto y comunicación E2E ejecutando un único comando (`docker compose up`).

11. Plan de trabajo



12. Estrategia de desarrollo

- **Flujo de Trabajo Git (GitFlow Simplificado):** Se establece una arquitectura de ramas estructurada para garantizar la estabilidad del código en producción. La rama **main** se reserva estrictamente para versiones estables y etiquetadas (releases). La rama **develop** actúa como el tronco central de integración continua. Todo desarrollo funcional nuevo se aísla en ramas secundarias con la nomenclatura **feature/<nombre-caracteristica>**, partiendo siempre de develop. Para la corrección urgente de errores en entornos avanzados, se habilitan ramas de tipo **hotfix/<incidencia>**.
- **Estrategia de Integración de Código (Pull Requests y Code Review):** Queda estrictamente prohibido el push directo sobre las ramas main y develop. La incorporación de nuevo código se realiza mediante Pull

Requests (PR) dirigidas hacia develop. Cada PR exige la revisión y aprobación obligatoria de al menos un integrante del equipo. Durante el proceso de Code Review, se evalúa el cumplimiento de estándares PEP 8, la legibilidad, la ausencia de deuda técnica y el correcto acoplamiento con los módulos existentes.

- **Validación de Contratos e Ingesta con Pydantic:** Para garantizar la integridad estricta de la información que ingresa al pipeline de inferencia, se definen esquemas de datos tipados mediante Pydantic en FastAPI. Estos modelos de datos actúan como un contrato de interfaz rígido, realizando validaciones automáticas de tipos, rangos numéricos permitidos y formatos de fecha en tiempo de ejecución. Ante datos corruptos o incompletos, el sistema rechaza la solicitud de inmediato devolviendo un código HTTP 422 (Unprocessable Entity), evitando la propagación de anomalías hacia los modelos de Machine Learning.
- **Testing Automatizado y Calidad con pytest:** El control de calidad del software se fundamenta en una suite de pruebas automatizadas desarrollada con el framework pytest. La cobertura abarca: (a) Pruebas unitarias sobre las transformaciones del pipeline de ETL (comprobando la correcta imputación de valores nulos y escalado de variables), (b) Pruebas de integración sobre los endpoints de FastAPI (verificando la coherencia de las respuestas HTTP 200 OK y la estructura JSON del output) y (c) Tests de carga de artefactos .joblib para constatar la viabilidad de la inferencia.

13. Riesgos técnicos actualizados

- **Riesgo 1: Fuga de Datos (Data Leakage) en el Feature Engineering**
  - **Descripción del problema:** Inclusión inadvertida de información temporal posterior a la salida del envío o métricas calculadas sobre el dataset global dentro del pipeline de entrenamiento.
  - **Impacto:** Alto. Genera una sobreestimación irreal de las métricas de rendimiento durante la fase de entrenamiento/validación y un colapso en la capacidad predictiva en producción.
  - **Probabilidad:** Media.
  - **Plan de mitigación técnico:** Auditoría estricta de variables en el pipeline de ETL para excluir atributos recopilados a posteriori. Configuración rigurosa de Pipelines de Scikit-Learn donde los transformadores (Scalers, Encoders) se ajusten exclusivamente

sobre el conjunto de entrenamiento (fit_transform) y solo se apliquen (transform) sobre validación y prueba.

- **Riesgo 2: Coste Asimétrico de Errores y Falsos Negativos**
  - **Descripción del problema:** Un umbral de decisión por defecto (0.50) puede no optimizar la detección de envíos críticos, generando falsos negativos de alto impacto por penalizaciones e incumplimientos de SLA.
  - **Impacto:** Alto. El modelo puede derivar hacia un sesgo óptimista que prediga sistemáticamente la clase mayoritaria, produciendo una alta tasa de falsos negativos inaceptable para la gestión operativa.
  - **Probabilidad:** Alta.
  - **Plan de mitigación técnico:** Aplicación de Threshold Tuning sobre la curva Precision-Recall para fijar un umbral optimizado que maximice el Recall (>80%) en la clase crítica y reduzca al mínimo los falsos negativos.
- **Riesgo 3: Latencia Elevada en las Respuestas Predictivas de la API**
  - **Descripción del problema:** Demoras en los tiempos de respuesta de la API debido a la lectura recurrente desde disco de los modelos .joblib o cuellos de botella en la fase de preprocesamiento.
  - **Impacto:** Medio. Afecta a la experiencia de usuario en el dashboard de Streamlit y degrada la capacidad del sistema para responder a llamadas de simulación interactiva.
  - **Probabilidad:** Media.
  - **Plan de mitigación técnico:** Carga de artefactos predictivos en memoria RAM durante el evento de arranque (lifespan startup) del servicio FastAPI. Optimización de la transformación de características vectorizando operaciones mediante NumPy/Pandas y aprovechando la arquitectura asíncrona (async/await) del servidor ASGI (Uvicorn).
- **Riesgo 4: Alta Cardinalidad en Variables Categóricas Geográficas y de Transporte**
  - **Descripción del problema:** Presencia de cientos de valores únicos en atributos clave como códigos postales, identificadores de rutas, hubs logísticos o transportistas.
  - **Impacto:** Medio. El uso indiscriminado de codificaciones como One-Hot Encoding dispara la dimensionalidad del dataset, saturando la memoria y desacelerando el entrenamiento.
  - **Probabilidad:** Alta.

    - **Plan de mitigación técnico:** Implementación de Target Encoding out-of-fold para variables de alta cardinalidad, o bien aprovechamiento del soporte nativo para características categóricas de LightGBM. Agrupamiento de categorías residuales o con baja frecuencia bajo la etiqueta común "Otros".
- **Riesgo 5: Incompatibilidad de Entornos y Conflictos de Dependencias**
    - **Descripción del problema:** Divergencias en las versiones de librerías (p. ej., Scikit-Learn, XGBoost, Pydantic v1 vs v2) entre los entornos locales de los distintos miembros del equipo.
    - **Impacto:** Alto. Puede provocar fallos catastróficos en la deserialización de artefactos .joblib o comportamientos erróneos no reproducibles al ejecutar la API.
    - **Probabilidad:** Alta.
- **Plan de mitigación técnico:** Uso estrictamente obligatorio de contenedores Docker basados en la misma imagen oficial de Python 3.10. Fijado riguroso de versiones exactas mediante un archivo requirements.txt bloqueado y orquestación unificada con Docker Compose.

14. Repositorio del proyecto

- **Repositorio Oficial:** https://github.com/Hulkobain/Grupo7-TFM-PlanetaFP
- **Estructura Modular del Proyecto:**

```
None
Grupo7-TFM-PlanetaFP/
├── docs/                # Entregables (PDFs de Fase 0, etc.)
├── data/                # Datasets crudos y procesados
├── environment/         # Archivos de entorno (requirements.txt,
Dockerfile)
├── src/
└── tests/ & README.md   # Pruebas unitarias y documentación
```